# Estruturas de dados para conjuntos disjuntos

## União-e-busca (*union-find*) e a análise do algoritmo de Kruskal

Jair

CMCC – UFABC

# Kruskal: onde aparecem os conjuntos disjuntos

Enquanto $S$ evolui de $\emptyset$ até uma árvore geradora, $(V, S)$ é sempre uma **subfloresta geradora** de $G$. Para $e = \{x, y\} \notin S$:

$$(V, S \cup \{e\}) \text{ é acíclico} \iff x \text{ e } y \text{ estão em subconjuntos disjuntos de } V.$$

Logo, o teste de aciclicidade do Kruskal equivale a verificar se $e$ não liga vértices do mesmo conjunto da partição; ao inserir $e$ em $S$, **unimos** o conjunto de $x$ com o de $y$.

$\Rightarrow$ precisamos de estruturas que representem e manipulem, dinamicamente, conjuntos disjuntos de modo eficiente: *union-find*.

# O problema e as operações

Representar dinamicamente uma família $\mathcal{F} = \{A_1, \ldots, A_k\}$ de subconjuntos disjuntos do universo $\mathcal{U} = \{1, \ldots, n\}$, em que cada conjunto é identificado por um **representante** (elemento do próprio conjunto). Enquanto um conjunto não for modificado por uma união, a resposta sobre o seu representante deve ser sempre a mesma.

`cria`$(x)$ para $x$ que ainda não pertence a nenhum conjunto, acrescenta $\{x\}$ à família $\mathcal{F}$;

`busca`$(x)$ devolve o representante do conjunto ao qual $x$ pertence;

`uniao`$(x, y)$ para $x$ e $y$ em conjuntos distintos $A_x$ e $A_y$, remove $A_x$ e $A_y$ de $\mathcal{F}$, acrescenta $A_x \cup A_y$ e escolhe um representante para ele.

**Como comparar as estruturas**

Supomos que as $n$ primeiras operações são `cria`, uma para cada elemento de $\mathcal{U}$, e analisamos o tempo total de uma **sequência de $m$ operações**. Em particular, $m \geq n$.

# Kruskal com conjuntos disjuntos

**Entrada:** grafo conexo $G = (V, E)$ com pesos nas arestas

1: $S \leftarrow \emptyset$
2: $F \leftarrow$ lista das arestas em ordem crescente de peso
3: **para cada** $v \in V$ **faça**
4: `cria`$(v)$
5: **para cada** $\{u, v\} \in F$ **faça**
6: **se** `busca`$(u) \neq$ `busca`$(v)$ **então**
7: insira $\{u, v\}$ em $S$
8: `uniao`$(u, v)$
9: **devolva** $(V, S)$

**Contagem das operações**

- $|V|$ operações `cria`;
- no máximo $2|E|$ operações `busca`;
- no máximo $|V| - 1$ operações `uniao`.

Total: menos que $2(|E| + |V|) = O(|E|)$, pois $G$ é conexo e $|V| \leq |E| + 1$.

**Consequência**

Com $m = O(|E|)$ operações, o custo do Kruskal é

$$O(|E| \log |E|) \text{ (ordenação) } + \text{ custo de } m \text{ operações.}$$

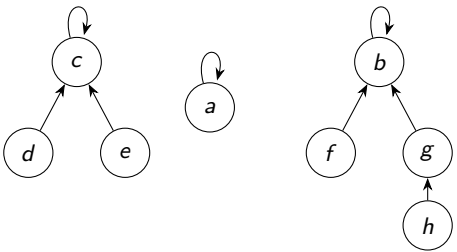
Como $|E| \leq |V|^2$, $O(|E| \log |E|) = O(|E| \log |V|)$. A estrutura escolhida decide se a ordenação domina.

# Floresta: cada conjunto é uma árvore

Cada elemento é um nó que aponta para o seu pai; a raiz é o representante e o pai da raiz é ela mesma.

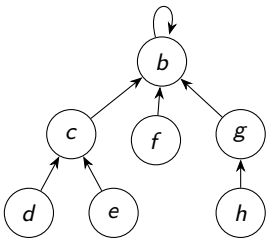
- `cria` cria, em tempo constante, uma árvore com um só nó;
- `busca`$(x)$ percorre o caminho de $x$ até a raiz, em tempo proporcional à profundidade de $x$, e devolve a raiz;
- a união de duas árvores é feita em tempo constante *a partir das duas raízes*, pendurando uma na outra.

Portanto, `uniao`$(x, y)$ custa duas buscas mais $O(1)$.




Representação com árvores de $\{c, d, e\}$, $\{a\}$ e $\{b, f, g, h\}$.

# Floresta: o problema e a união por tamanho




Resultado de `uniao`$(e, f)$: a raiz $c$ (tamanho 3) é pendurada na raiz $b$ (tamanho 4).

Essa estrutura pode ser muito ruim: se pendurarmos sempre a raiz da árvore maior numa árvore unitária, formamos uma cadeia com $n$ nós, em que cada busca no nó mais profundo custa $\Theta(n)$.

# Floresta: o problema e a união por tamanho

O *tamanho* de uma árvore é o seu número de nós.

A *altura* de uma árvore é a maior distância de um nó até a raiz.

**União por tamanho**

A raiz da árvore de menor tamanho é pendurada na raiz da árvore de maior tamanho (em caso de empate, em qualquer uma).

Qual a relação entre tamanho e altura após uma sequência de uniões com essa heurística?

# Floresta: algoritmos com união por tamanho

Cada nó tem $\mathrm{pai}[x]$ e, nas raízes, $t[x]$ = número de elementos da árvore enraizada em $x$.

**cria**$(x)$

1: $\mathrm{pai}[x] \leftarrow x$
2: $t[x] \leftarrow 1$

**busca**$(x)$

1: **enquanto** $x \neq \mathrm{pai}[x]$ **faça**
2: $\quad x \leftarrow \mathrm{pai}[x]$
3: **devolva** $x$

**link**$(x, y)$

1: $t[x] \leftarrow t[x] + t[y]$
2: $\mathrm{pai}[y] \leftarrow x$

`link` pressupõe que $x$ e $y$ são raízes distintas com $t[x] \geq t[y]$ (garantido por `uniao`); `uniao` pressupõe que $x$ e $y$ estão em árvores distintas.

**uniao**$(x, y)$

1: $r_x \leftarrow$ `busca`$(x)$; $r_y \leftarrow$ `busca`$(y)$
2: **se** $t[r_x] \geq t[r_y]$ **então**
3: $\quad$ `link`$(r_x, r_y)$
4: **senão**
5: $\quad$ `link`$(r_y, r_x)$

# União por tamanho: tamanho × altura

**Proposição**

Usando união por tamanho, e supondo que as $n$ primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos $2^h$ nós, onde $h$ é a sua altura.

# União por tamanho: tamanho × altura

**Proposição**

Usando união por tamanho, e supondo que as $n$ primeiras operações são `cria`, em qualquer momento cada árvore tem pelo menos $2^h$ nós, onde $h$ é a sua altura.

**Prova.**

Indução no número $k$ de uniões já realizadas. Para $k = 0$, as únicas operações até então que alteram a estrutura são `cria`, e toda árvore tem um nó e altura 0, com $1 \geq 2^0$. Seja $k \geq 1$: como as $n$ primeiras operações são `cria`, nada além das uniões altera a estrutura. A $k$-ésima união liga duas árvores distintas $A$ e $B$, com tamanhos $t_A, t_B$ e alturas $h_A, h_B$; por indução $t_A \geq 2^{h_A}$ e $t_B \geq 2^{h_B}$. Sem perda de generalidade $t_A \geq t_B$ e a raiz de $B$ é pendurada na de $A$. A árvore resultante tem $t' = t_A + t_B$ nós e altura $h' = \max\{h_A, h_B + 1\}$.

- Se $h' = h_A$: $t' \geq t_A \geq 2^{h_A} = 2^{h'}$.
- Se $h' = h_B + 1$: $t' = t_A + t_B \geq 2t_B \geq 2 \cdot 2^{h_B} = 2^{h'}$.

□

# União por tamanho: custo

**Corolário**

Usando união por tamanho, com as $n$ primeiras operações `cria`, o tempo de $m$ operações é $O(m(1 + \log n))$.

**Prova.**

Pela proposição, uma árvore de altura $h$ tem $2^h \leq n$ nós, logo $h \leq \log_2 n$ e cada `busca` custa $O(1 + \log n)$. Cada `cria` custa $O(1)$ e cada `uniao` custa duas buscas mais $O(1)$, isto é, $O(1 + \log n)$. O total é $O(n + m(1 + \log n)) = O(m(1 + \log n))$, pois $m \geq n$. □

**Kruskal com floresta**

Com $m = O(|E|)$ e $n = |V|$, o custo das operações é $O(|E| \log |V|)$; somado à ordenação, o tempo total é $O(|E| \log |V|)$.

# União por altura e por *rank*

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura.

# União por altura e por *rank*

Como o tempo de `busca` é proporcional à altura da árvore, parece mais natural uma heurística baseada na altura.

**União por altura**

A raiz da árvore mais baixa é pendurada na raiz da árvore mais alta (em caso de empate, em qualquer uma).

Cada raiz $r$ guarda $h[r]$, a altura da árvore enraizada em $r$, iniciada com 0. `link` pressupõe raízes distintas com $h[x] \geq h[y]$.

**cria**$(x)$

1: $\mathrm{pai}[x] \leftarrow x$
2: $h[x] \leftarrow 0$

**link**$(x, y)$ **(por altura)**

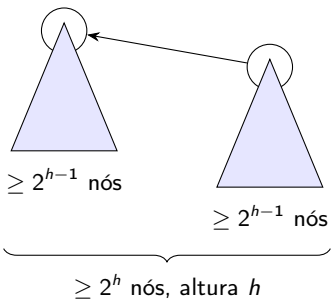
1: **se** $h[x] = h[y]$ **então**
2: $\quad h[x] \leftarrow h[x] + 1$
3: $\mathrm{pai}[y] \leftarrow x$

**uniao**$(x, y)$

1: $r_x \leftarrow$ `busca`$(x)$;
2: $r_y \leftarrow$ `busca`$(y)$
3: **se** $h[r_x] \geq h[r_y]$ **então**
4: $\quad$ `link`$(r_x, r_y)$
5: **senão**
6: $\quad$ `link`$(r_y, r_x)$

# Floresta: por que a altura fica pequena?

**Ideia:** a altura só aumenta quando pendurada uma árvore de *mesma altura*. Portanto, uma árvore de altura $h$ foi montada a partir de **duas de altura** $h-1$:




Logo uma árvore de altura $h$ tem $\geq 2^h$ nós; com no máximo $n$ nós, a altura é $\leq \log_2 n$. Cada `busca` custa $O(\log n)$ e

# União por altura e por *rank*

Valem os mesmos resultados:

**Proposição**

Usando união por altura, e supondo que as $n$ primeiras operações são `cria`, em qualquer momento cada árvore tamanho$\geq 2^h$ nós, onde $h$ é a sua altura.

**Corolário**

Usando união por altura, com as $n$ primeiras operações `cria`, o tempo de $m$ operações é $O(m(1 + \log n))$.

# Busca com compressão de caminhos e união por *rank*

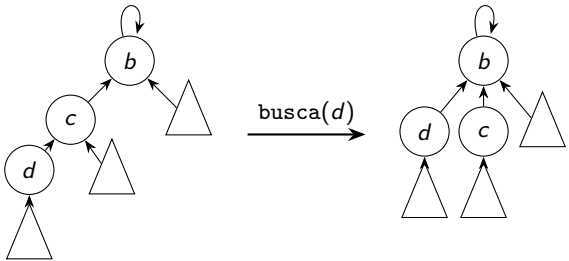
Quando introduzirmos compressão de caminhos, $h$ deixa de ser a altura exata: passa a se chamar **rank**.

# Busca com compressão de caminhos e união por *rank*

Quando introduzirmos compressão de caminhos, $h$ deixa de ser a altura exata: passa a se chamar **rank**.

## Compressão de caminhos

Quando fazemos `busca`$(x)$, aproveitamos o percurso de $x$ até a raiz e desviamos os ponteiros `pai`$[y]$, de todo $y$ nesse caminho, para a raiz; na união penduramos a árvore de menor *rank* na raiz da de maior *rank*.




Esquema de uma busca com compressão de caminhos: $d$ e $c$ passam a ser filhos da raiz $b$.

# *Rank* e os algoritmos

A compressão encurta caminhos, e manter a altura exata a cada busca seria caro. Em vez disso guardamos em cada raiz um **rank**:

- começa em 0 e só muda numa união: se as duas raízes têm o mesmo rank, a que recebe sobe 1;
- é uma **cota superior** da altura (igual a ela quando não há compressão);
- um nó de rank $r$ tem pelo menos $2^r$ descendentes, como no argumento da altura: então $\mathrm{rank} \leq \log_2 n$.

**Duas propriedades que usaremos**

(a) Todo ancestral próprio de $x$ tem rank **estritamente maior** que o de $x$.

(b) Depois que deixa de ser raiz, o rank de um nó **nunca mais muda**; o rank do *pai* de $x$ só pode aumentar (ou ficar igual) ao longo do tempo.

# *Rank* e os algoritmos

**cria**$(x)$

1: $\text{pai}[x] \leftarrow x$
2: $\text{rank}[x] \leftarrow 0$

**busca**$(x)$ **com compressão**

1: **se** $x \neq \text{pai}[x]$ **então**
2: $\quad \text{pai}[x] \leftarrow \texttt{busca}(\text{pai}[x])$
3: **devolva** $\text{pai}[x]$

**link**$(x, y)$

1: **se** $\text{rank}[x] = \text{rank}[y]$ **então**
2: $\quad \text{rank}[x] \leftarrow \text{rank}[x] + 1$
3: $\text{pai}[y] \leftarrow x$

**uniao**$(x, y)$

1: $r_x \leftarrow \texttt{busca}(x)$; $r_y \leftarrow \texttt{busca}(y)$
2: **se** $\text{rank}[r_x] \geq \text{rank}[r_y]$ **então**
3: $\quad \texttt{link}(r_x, r_y)$
4: **senão**
5: $\quad \texttt{link}(r_y, r_x)$

`link` pressupõe que $x$ e $y$ são raízes distintas com $\text{rank}[x] \geq \text{rank}[y]$ (garantido por `uniao`), e `uniao` que $x$ e $y$ estão em árvores distintas.

# O resultado de Tarjan e a função $\alpha$

**Teorema (Tarjan, 1975)**

Com união por rank e compressão de caminhos, o tempo gasto com $m$ operações num universo de tamanho $n$, com $m \geq n$ (as $n$ primeiras sendo `cria`), é $O(m\,\alpha(m,n))$.

Não demonstraremos esse teorema (veja CLR); o resultado que mostraremos é um pouco menos preciso.

A função de Ackermann (versão de CLR), para $i, j \geq 1$:

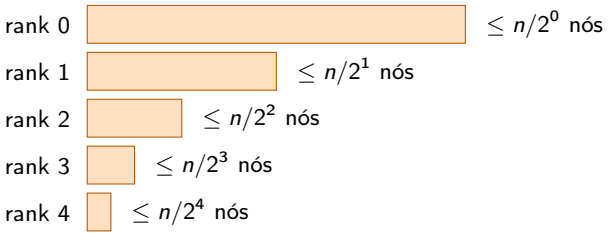
$$A(i,j) = \begin{cases} 2^j & \text{se } i = 1, \\ A(i-1,2) & \text{se } i \geq 2 \text{ e } j = 1, \\ A\big(i-1, A(i,j-1)\big) & \text{se } i \geq 2 \text{ e } j \geq 2, \end{cases} \qquad \alpha(m,n) = \min\big\{ i \geq 1 \colon A(i, \lfloor m/n \rfloor) > \log_2 n \big\}.$$

$A(2,j)$ é uma torre de exponenciais com $j+1$ dois, $A(3,1) = 16$ e $A(4,1) = A(2,16)$ é uma torre de 17 dois, imensamente maior que $10^{80}$. Como $A$ é crescente em $j$, $\alpha(m,n) \leq \alpha(n,n)$, e portanto $\alpha(m,n) \leq 4$ para todo $n < 2^{A(4,1)}$.

| $n$ | $\alpha(n,n)$ |
|---|---|
| $1-3$ | 1 |
| $4-15$ | 2 |
| $16-65535$ | 3 |
| $65536-2^{A(4,1)}-1$ | 4 |

# Análise (1/3): poucos nós têm rank alto

Um nó de rank $r$ "traz junto"[1] pelo menos $2^r$ elementos, e esses elementos **não são reaproveitados** por outro nó do mesmo rank. Logo, há no máximo $n/2^r$ nós de rank $r$:




É uma **pirâmide**: metade dos nós tem rank baixo, e os ranks altos são raros. Em particular, os nós de rank maior que $j$ são no máximo $n/2^j$.

[1] árvore **no momento em que ele deixa de ser raiz** (depois disso a compressão pode encolher a subárvore, mas esse conjunto fica fixo).

# Análise (2/3): agrupar os ranks em faixas

Considere $m$ operações, as $n$ primeiras `cria`, com união por *rank* e compressão. Sejam

$$b_0 = 1 \text{ e } b_g = 2^{b_{g-1}}$$

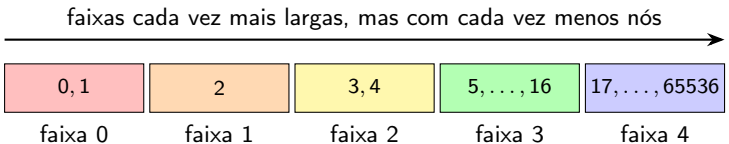
isto é, $1, 2, 4, 16, 65536, \ldots$

Particionamos $\mathcal{U}$ em **grupos** segundo o *rank final*:

- Grupo 0: elementos de rank 0 e 1;
- Grupo 1: elementos de rank em $(1, 2]$;
- Grupo 2: elementos de rank em $(2, 4]$;
- Grupo 3: elementos de rank em $(4, 16]$;
- Grupo 4: elementos de rank em $(16, 65536]$;
- $\vdots$
- Grupo $g \geq 1$: elementos de rank em $(b_{g-1}, b_g] = (K, 2^K]$.

# Análise (2/3): agrupar os ranks em faixas

Dividimos os ranks possíveis $(0, \ldots, \log_2 n)$ em **faixas** que ficam cada vez mais largas: a faixa $g$ cobre os ranks $(K, 2^K]$, com $K$ o fim da faixa anterior.




A largura **explode** (de $K$ para $2^K$), então bastam $\lg^* n$ faixas para cobrir todos os ranks.

**Compromisso:** faixa mais larga $\Rightarrow$ um nó dela pode ser contado em mais buscas (o pai dele passa por mais ranks sem sair da faixa), mas a pirâmide garante que há menos nós nela.

# Análise (3/3): que nós são contados em cada busca?

O custo de uma busca é o número de nós do caminho percorrido: **contamos $O(1)$ para cada nó**. Separamos os nós do caminho em dois tipos e limitamos quantas vezes cada tipo pode ser contado:

## Tipo 1: a raiz, o filho da raiz, ou quem **troca de faixa** para o pai

No caminho o rank só cresce, então o caminho cruza no máximo $\lg^* n$ faixas: $\leq \lg^* n + 1$ **nós por busca**, em $O(m \lg^* n)$ ao todo.

## Tipo 2: $y$ e o pai estão na **mesma** faixa

Cada vez que $y$ é contado como Tipo 2, ele passa a apontar para a raiz, que tem rank **maior** que o do pai anterior. Como a faixa tem no máximo $2^K$ ranks, $y$ é contado no máximo $2^K$ vezes e depois seu pai sai da faixa. A faixa tem no máximo $n/2^K$ nós:

$$\underbrace{\frac{n}{2^K}}_{\text{nós}} \cdot \underbrace{2^K}_{\text{vezes contado por nó}} = n \qquad \Rightarrow \qquad \text{cada faixa custa } n.$$

Com $\lg^* n$ faixas: $O(n \lg^* n)$.

# Resumo: $m$ operações ($n$ primeiras `cria`) e Kruskal

| **Estrutura** | **Tempo de $m$ operações** | **Kruskal** |
|---|---|---|
| Vetor | $O(m + n^2)$ | $O(\|E\| \log \|E\| + \|V\|^2)$ |
| Listas ligadas + menor lista | $O(m + n \log n)$ | $O(\|E\| \log \|V\|)$ |
| Floresta, união por tamanho/altura | $O(m(1 + \log n))$ | $O(\|E\| \log \|V\|)$ |
| Floresta, rank + compressão | $O((m + n) \lg^* n)$ (Tarjan: $O(m\,\alpha(m, n))$) | $O(\|E\| \log \|V\|)$ |

Com listas ou florestas, a ordenação das arestas passa a dominar o custo do Kruskal; a estrutura de dados deixa de ser o gargalo.